BombLab 实验报告

一、实验说明

本实验通过每关提供的 64 位 `challenge` 计算输入整数，而不是记忆某个 seed 对应的固定答案。分析时以 `bomb` 的反汇编为依据，重点确认参数传递、32 位寄存器运算、循环/递归、跳转表、链表及函数指针表的行为。无符号 32 位运算按模 \(2^{32}\) 回绕。

二、实验过程

（1）Phase 1：调用与返回值

`phase_01` 要求一个无符号整数。它把 `challenge` 传给 `p1_compute`，后者拆出低 32 位 `lo`、高 32 位 `hi`，以及右移 16 位后的低 32 位 `mid16`，依次调用三个步骤函数。

- `p1_step2` 的 `lea (%rdi,%rsi,2)` 表示 `hi + 2*lo`，随后与 `0x4587` 异或。

- `p1_step0` 的 `sub` 和 `lea 0x9ed5(..., ..., 4)` 合起来表示 `5*(lo-mid16)+0x9ed5`。

- `p1_step1` 先加 `0xb2fb`，再与上一步的结果异或。

因此令 `s2=(hi+2*lo)^0x4587`、`s0=5*(lo-mid16)+0x9ed5`，最终答案为 `(s2+0xb2fb)^s0`。



（2）Phase 2：六项递推

`phase_02` 用六个 `%u` 读取六个无符号整数。首项的汇编先将 challenge 右移 11 位、乘 2，再以 `movzbl %al,%eax` 只保留低 8 位，最后加 20。因此：

`x[0] = (((challenge >> 11) * 2) & 0xff) + 20`

等价写法是 `((challenge >> 10) & 0xfe) + 20`。

循环索引 `i` 从 1 到 5。移位计数从 22 开始、每次增加 3；每轮取一个 4 位值 `n=(challenge >> (22+3*(i-1)))&0xf`，先计算 `n+i`，与前一项异或，再乘 3：

`x[i] = 3 * ((n+i) ^ x[i-1])`



（3）Phase 3：switch 与跳转表

本关读入两个整数。索引是 `idx=challenge&7`；传给分支函数的第二个参数是 `arg=(challenge>>32)^(challenge>>3)` 的低 32 位。不能按 `p3_case_0` 到 `p3_case_7` 的函数地址顺序猜索引关系，应读取跳转表地址并把每个表项对应到实际 case。

| idx | 实际分支 | 返回表达式 |

|---:|---|---|

| 0 | `p3_case_0` | `(arg*4)^0xbd1` |

| 1 | `p3_case_7` | `(arg*2)^0x6e72` |

| 2 | `p3_case_3` | `(arg^0x633)+(arg>>2)` |

| 3 | `p3_case_2` | `arg*5+0x7d5` |

| 4 | `p3_case_5` | `(arg^0x8dbd)+(arg>>1)` |

| 5 | `p3_case_6` | `arg*5+0x3a4d` |

| 6 | `p3_case_1` | `arg*5+0x4189d` |

| 7 | `p3_case_4` | `arg*5+0x181a5` |

输入的第二个整数等于该分支的返回值。



（4）Phase 4：递归与返回过程

本关输入一个整数。初始递归区间是 `[0,127]`，目标值 `target=challenge&0x7f`，辅助参数 `salt=(challenge>>7)^(challenge>>32)`（按 32 位使用）。每层计算中点 `mid=low+(high-low)/2`。

- 若 `mid==target`，递归基例返回 `(mid+0xd6)^salt`。

- 若 `target<mid`，递归到 `[low,mid-1]`；返回时将子调用结果乘 5，与 `mid+salt` 异或，再与 `0x8` 异或。

- 否则递归到 `[mid+1,high]`；返回时执行同样的乘 5 和 `mid+salt` 异或，再与 `0x1d` 异或。

汇编中的递归 `call p4_walk` 和 `leave`/`ret` 对应“先到基例，再逐层组合返回值”的过程。



（5）Phase 5：结构体与链表遍历

本关读取两个整数。炸弹中的节点可按两个 32 位字段和一个 next 指针理解：低 32 位 `low`、高 32 位 `high`、后继指针。challenge 的低 3 位选择一个头节点；`(challenge>>3)&3` 决定从头节点开始继续访问多少个后继节点。

初值为 `v=((challenge>>6)^(challenge>>32))^0xc96`。对头节点及随后访问的节点，逐个执行：

`v=(v+low)*5 ^ high`

返回结果是第一个答案；遍历过程中最后访问节点的 `high` 被写入输出指针，是第二个答案。该过程也说明了为什么不能把节点数臆定为固定的 3 个：访问次数取决于 challenge。



（6） Phase 6：函数指针表与间接调用

本关输入一个整数。起始值为 `v=(challenge>>6)^(challenge>>32)` 的低 32 位。challenge 的低 3 位从函数指针表选出第一个操作；再右移 3 位取低 3 位，选出第二个操作。

汇编先执行 `p6_ops[challenge&7](v)`，将结果与 `0xe0b2` 异或，再执行 `p6_ops[(challenge>>3)&7]`，最后加 `0xf263`。函数指针表索引到实际操作的映射为：

`[op2, op1, op0, op5, op2, op3, op3, op4]`

其中 `op0` 至 `op5` 对应移位/XOR、加法/XOR、旋转等具体运算；每个操作按 32 位值计算。这个表的映射必须按 `.rodata` 中的指针顺序读取，不能只根据函数名或地址顺序推断。



（7）Secret（Honor）

Secret 输入行除三个整数外还包含门禁前缀。`secret_gate` 检查输入缓冲区偏移 32 处的 `OPEN`，再从偏移 36 处交给 `secret_phase` 解析三个整数。因此求解器输出行以 32 个空格和 `OPEN` 开始，之后跟三个无符号十进制数。

令 `cl/ch` 为 challenge 的低/高 32 位，`tl/th` 为 context 的低/高 32 位，并令 `a=cl^ch`、`b=tl^th`：

- `x=tl&3`。

- `secret_mix(q)` 对应 `rol32(q,7)`、加 `0x935d`、再与 `0x5de2` 异或。

- 第二个数 `y=secret_mix(a^b)^((x*0x935d)^(context>>16))`。

- 第三个数 `z=y^((a>>7)+b+x+0x5de2)`。

以上各步都按 32 位无符号运算理解。输出缓冲区容量通过 `snprintf` 的返回值检查，避免截断后仍报告成功。

```
elngxu@LAPTOP-D640PIUT:~/ics-2026-bomblab$ set -o pipefail; { ./defuser --seed 1234567890; ./defuser --seed 1234567890 --secret; } | ./bomb --[illegible]
BOMBLAB 2026                                                    这串还没有加大
```

```
          *
          |
      .-""""""-.
    .'          '.
   /    LOOP     \
   |              |
   \              /
    '.          .'
      '-......-'
```

```
[1 当前] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]
```

一个普通的夜晚，你和林宴在实验室熬上了一台旧设备，
却收入了一座拥生的异常城市，打在这里的，是 AI。
初识的惊喜随后开始，考验正在为你带来一场灯。
而在前面，指挥监督门开门后，重新关闭，台门将锁。
解密：阅读一条资讯：

  投资到这里还会有人来，
  保你们好运，
  — sky

窗口每客留着*级同类给全*的按钮。
林宴把外套搭在椅背上：“先看看。”
他不用按，上面有一张点亮，表示有两次提示机会给予帮助。
它不是一张普通的数，对是你的每个谜题都有一次。
第 1 关 · 维护舱
终端上出现一份维护记录。
你输了一下两年，屏幕亮起，六个数字只有第一个在闪。
林宴把资讯内每字先读全：“我看见了，你看程序。”
终端继续列表入如果，但这没有改行，旧的口令工具已经没不到了。
窗下去到维修员的八号一个真正的输入。

Phase 1 challenge: 0x85236bd7f08928b
Input (unsigned decimal integers):
Phase 1 defused.

门锁解了一声，维护舱开始接收新的记录。
以上是我从未到过的不有时，城市的旧都起始结构如下：
对这一份东西记录上问题基础的修改，修复记下解锁，推开门。

[1 已解除] [2 当前] [3 待解除] [4 待解除] [5 待解除] [6 待解除]

第 2 关 · 除楼
神经没有钟声，内轮照常转动，日期却没有前天。
林宴翻开纸维里的记录：同一天的几次检时被写在不同位置。
每一次都责任！一次的区别。你们每次确认这两记录被报告率。
时钟的速度每又被迫相角，动机还是刚才新的读：
“今天来不及了，明天一定识别。”

Phase 2 challenge: 0x1688d3ec6fd448cd
Input (unsigned decimal integers):
Phase 2 defused.

终端检查通过了，终楼监出一张新记录，几秒后又变回原样的一张。
“不是神保护，”林宴把两张记录放在一起，“这是出去又回来了。”
你们当前已经在上层，重要的地址通过新闻都已经的资讯。

[1 已解除] [2 已解除] [3 当前] [4 待解除] [5 待解除] [6 待解除]

第 3 关 · 公监控
通往外楼的线路，你们来到第口的旁通站。
墨设门前边来对这一个路口，又被这两扇一个旅山。
林宴在上面好了一张解包数据，“这一层有旧地址之间题，”
地里仅是它几种不同的处理办法，请求图标记法定会走哪一条。

Phase 3 challenge: 0x2073eb8e2e8c584
Input (unsigned decimal integers):
Phase 3 defused.

这次没求没有再每两来，分流站的石这是内室锁，要到一条两层：
“来说到分层公位，”他告后面有一份旧日志，
林宴看好底程程，你们学习马格，档案馆的门刚刚打开。

[1 已解除] [2 已解除] [3 已解除] [4 当前] [5 待解除] [6 待解除]

第 4 关 · 档案馆
馆客很里没有人，书架上的指示灯一层层两两地亮。
每有馆序两份这九次报告，要分有不同的日志类。
你们找到入口，后面的数据还要负责旧头与原层两下去。
林宴把对于的读题抄写出：“这次提与两份号码，报警记录走。”

Phase 4 challenge: 0x5a2f6ab296b046d89
Input (unsigned decimal integers):
Phase 4 defused.

最后一层来引道门了一份完整的档，还有旅次迁移到上的记录。
最设备很上方，公报没有做完，城市只能负责恢复最后一份旧存档：
地区与大部以前走，数了要避免部区了。
你对后面的每九层迁移每对的设备，你认出了维修通服装。

[1 已解除] [2 已解除] [3 已解除] [4 已解除] [5 当前] [6 待解除]

第 5 关 · 巡检回廊
巡检回廊连着只要维修有区，钥！的设备报销与指令报修序。
这层楼从一台两已经封走，报到另一楼的至个检门。
每个锁的左与数，通过受对的礼当前要位。
她冲下船两报条：“今天值好换门，明天装！，”
林宴认出那是灯：“我们越多来时，他就在前没个。”

Phase 5 challenge: 0x71a21dc9f12f89c7
Input (unsigned decimal integers):
Phase 5 defused.

最后一处连接也到上了，今天的旧各就也当一次进入待保存的记录。
你把最后报证两序了一下灯体，灯亮了一排，又被快小心地又掉，
门缓到两次至时，“地点，
交换道中！只有最后一项，你们没有记录的两两都失。

[1 已解除] [2 已解除] [3 已解除] [4 已解除] [5 已解除] [6 当前]

第 6 关 · 交接室
交接室的灯还亮着，桌上有着一份没有签的记录。
你们刚才报到的那两，都是，你们认会有状态都到了这里。
林宴把记录推了一下灯体，灯亮了一排，又被快小心地又掉，
门缓到两次至时，“地点，
交换道中！只有最后一项，你们没有记录的两两都失。

Phase 6 challenge: 0x73eea1886fcda42b
Input (unsigned decimal integers):
Phase 6 defused.

终端换项这份记录，最后一项在“保存完成”，
林宴没有立！起身，你们等了一会儿，屏幕！但自动向动走了一天。
这次没有报告任何。

Congratulations! You've defused the bomb.
[hint] 小提醒：下一层更深，留意时间与顺序。

你们走回电！，通往动机已经发动了车：
“你不敢留了。今天两层点点，”
他的脸的边表好了，三张止息的长与若“期末签上”的张张阿卜来，
林宴把记录收了，决定带好了一张片，装上了你。

Honor · 通过名册系统

林宴准备关闭档案员员，忽然指着员卜面一张：“这里还有个名称，”
文件夹叫“阿告两次的人”，名行是 sky 和旧层。
最后写着：“交报完成后，如果你还有时间，可以到船舰这里来。”
这份旧两字人是马城市的负责，你们发现楼梯通上通城台，
走上的通边楼的台号在旧两序该可的输入，
标市已经安全了，至已继续留下表，也你们自己决定。

Secret phase defused.
[hint] 秘密的关卡后，那里的发已恢复。

阅读器亮起来，是先出现的不是代码，是两份很早的留言。

  旧港：

  北方运边传通知到了，让许有人，
  船了项目我们被迫退出了。你得我看出得的时候，我还很不一定决得上，
  就在鑫里走了，又也有点不习惯，以后最到问题，
  不能再的每个到你旁边，问一句“你真这样不算有东病”。

  你想留下整修手里的事，还记得你为它准备了多久，
  我出现真实，最后留给谁什么样，
  因是这两天前几小，我把记录整理好了，
  你有举报点，报给先生自己位置。

  走之前出去给我说句，这次原在实验室满合了，

  — sky

留了几天，旧港在后面添了一份回如，

  你报当时间等定，

  这里的东西其实很有，你在北方有了新基础，也可以回来做，
  我真不下止，这边的事那台这内，我们欠自社助活，
  以后你不能会在上面，到时候当东样，你也一样，
  如同为申请的报每很重定，遇到不许你的事情不好意思讲。

  施 给上的时间，我想石石你 直装的北方。

  — 许洪

后面是这些生新新很很的更新，
  北京方的当用，一次接了万州的项目，一经地了很久才修好的代码，
  sky 人了北方，还是两年这样，我们欠自社助活，
  时间会加速了好几天，很会很多认识的朋友，
  有一个，还是以为城市修理事的加过，后来 sky 也换了工作，
  这种城得过个，好时长，城都年度一次交两，

最后的只是 sky 给的，两句话了其许远后来集来的，
最后一张便签贴在灯柱旁边：

  “下次同来，先中一会儿，”

林宴拉上背中，起身了，你们站了一会儿，没有再做记录，
你下来起今天的新对，

离开楼城台时，你想起自己收到了很久的一份项目介绍，
那比原进调又的动力，林宴也在自己的本子上记了几行，

返出前，终端收到 sky 的资讯：
“旧港都到了，城市很正常，谢谢你们，期下回维的我来接，”

实验室的灯还亮着，外头的冷的雨下，你确认又件都保存了，关上出门，
林宴按了按提搜：“许立意，整体比当可定，”
你把每个新同学上：“新气象的，先解题吧，”
你们关掉灯，走出了实验室。

谢谢你远道而来，与我相遇。

顺安，

— sky

```
elngxu@LAPTOP-D640PIUT:~/ics-2026-bomblab$ make test
```

三、验证记录

公开 seed 来自 `tests/public.json`：`0, 1, 7, 42, 99, 123, 2026, 65535, 987654321, 18446744073709551615`。

| 验证内容 | 命令 | 结果 |

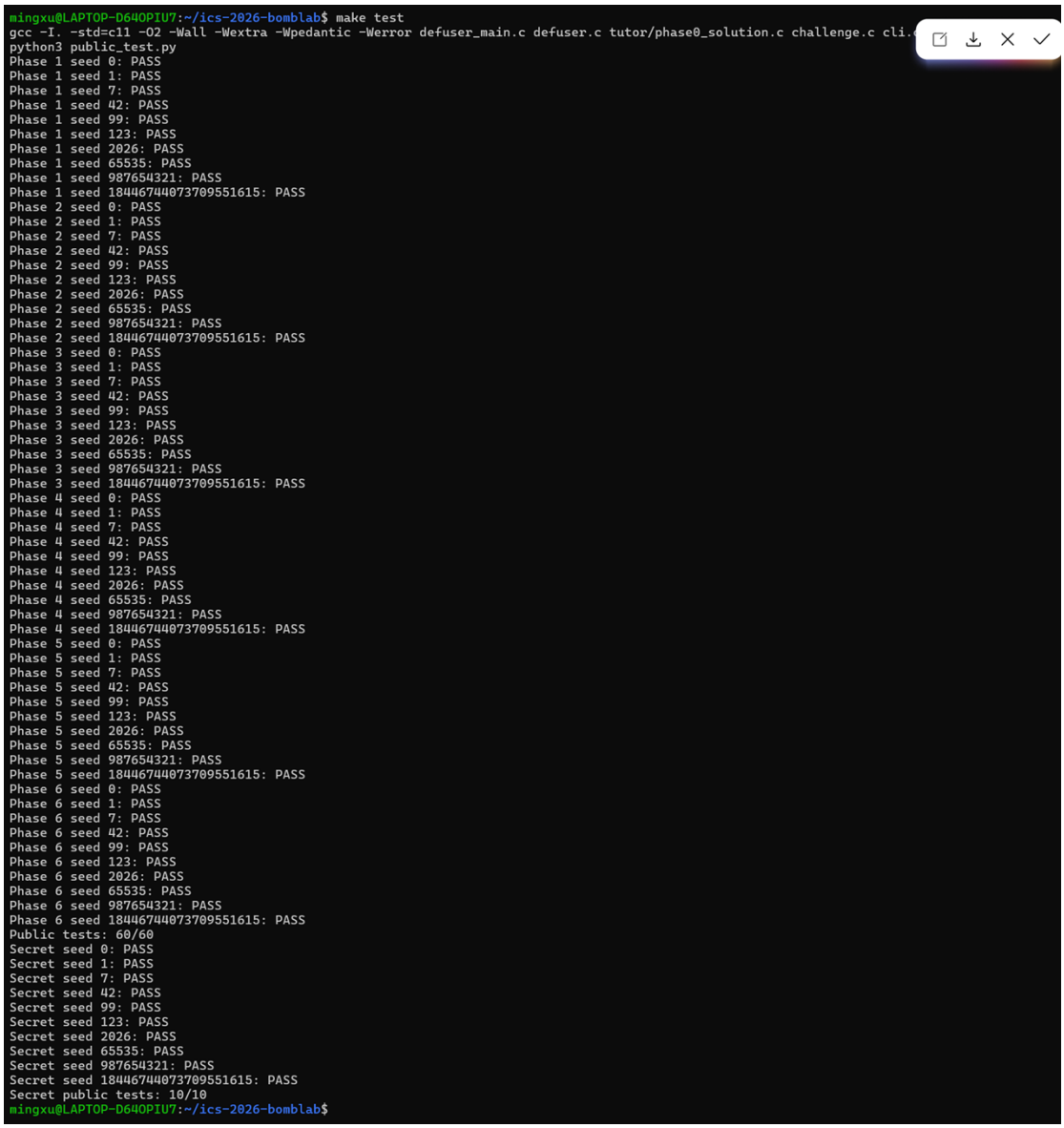
|---|---|---|

| Phase 1–6 公开测试 | `python3 public_test.py` | 60/60 通过 |

| Secret 公开测试 | `python3 public_test.py --secret` | 10/10 通过 |

| Phase 1–6，非公开 seed 1234567890 | `./defuser --seed 1234567890 --phase N \| ./bomb --seed 1234567890 --phase N`，`N=1..6` | 每关均显示对应 `Phase N defused.` |

| Secret 完整流程，非公开 seed 1234567890 | `{ ./defuser --seed 1234567890; ./defuser --seed 1234567890 --secret; } \| ./bomb --seed 1234567890` | 六关及 Secret 均通过，显示 `Secret phase defused.` |



四、**使用的资料与辅助工具**

- 仓库 `README.md`：实验接口、测试方式和报告格式要求。

- 本地 `bomb` 的反汇编与符号：使用 `gdb`、`objdump`、`nm` 检查函数、跳转表、结构体字段及间接调用。

- 本地 `defuser.c`、`challenge.c` 和 `tests/public.json`：核对求解实现、challenge/context 生成方式及公开 seed。

- GCC 与 `make`：使用项目规定的 `-Wall -Wextra -Wpedantic -Werror` 编译。

- `python3 public_test.py`：运行公开测试。

- Copilot AI 助手：用于解释部分汇编、定位实现与汇编不符之处、协助调试及核对测试结果。代码和报告内容应由本人复核并确保能够解释。